In [1]:
```
import torch
import numpy as np
import random

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
```

Out[1]:
```
<torch._C.Generator at 0x104f1e3f0>
```

In [2]:
```
import warnings

warnings.filterwarnings("ignore")
```

In [3]:
```
import sys
!{sys.executable} -m pip install torch "gluonts[torch]"
```

```
Requirement already satisfied: torch in /opt/miniconda3/envs/econ491_walmart
_forcasting/lib/python3.10/site-packages (2.11.0)
Requirement already satisfied: gluonts[torch] in /opt/miniconda3/envs/econ49
1_walmart_forcasting/lib/python3.10/site-packages (0.16.2)
Requirement already satisfied: filelock in /opt/miniconda3/envs/econ491_walm
art_forcasting/lib/python3.10/site-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /opt/miniconda3/
envs/econ491_walmart_forcasting/lib/python3.10/site-packages (from torch)
(4.15.0)
Requirement already satisfied: setuptools<82 in /opt/miniconda3/envs/econ491
_walmart_forcasting/lib/python3.10/site-packages (from torch) (81.0.0)
Requirement already satisfied: sympy>=1.13.3 in /opt/miniconda3/envs/econ491
_walmart_forcasting/lib/python3.10/site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /opt/miniconda3/envs/econ4
91_walmart_forcasting/lib/python3.10/site-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /opt/miniconda3/envs/econ491_walmar
t_forcasting/lib/python3.10/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /opt/miniconda3/envs/econ491
_walmart_forcasting/lib/python3.10/site-packages (from torch) (2025.12.0)
Requirement already satisfied: numpy<2.2,>=1.16 in /opt/miniconda3/envs/econ
491_walmart_forcasting/lib/python3.10/site-packages (from gluonts[torch])
(2.1.3)
Requirement already satisfied: pandas<3,>=1.0 in /opt/miniconda3/envs/econ49
1_walmart_forcasting/lib/python3.10/site-packages (from gluonts[torch])
(2.3.3)
Requirement already satisfied: pydantic<3,>=1.7 in /opt/miniconda3/envs/econ
491_walmart_forcasting/lib/python3.10/site-packages (from gluonts[torch])
(2.12.5)
Requirement already satisfied: tqdm~=4.23 in /opt/miniconda3/envs/econ491_wa
lmart_forcasting/lib/python3.10/site-packages (from gluonts[torch]) (4.67.3)
Requirement already satisfied: toolz~=0.10 in /opt/miniconda3/envs/econ491_w
almart_forcasting/lib/python3.10/site-packages (from gluonts[torch]) (0.1
2.1)
Requirement already satisfied: lightning<2.5,>=2.2.2 in /opt/miniconda3/env
s/econ491_walmart_forcasting/lib/python3.10/site-packages (from gluonts[torc
h]) (2.4.0)
Requirement already satisfied: pytorch-lightning<2.5,>=2.2.2 in /opt/minicon
da3/envs/econ491_walmart_forcasting/lib/python3.10/site-packages (from gluon
ts[torch]) (2.4.0)
Requirement already satisfied: scipy<1.16.0 in /opt/miniconda3/envs/econ491_
walmart_forcasting/lib/python3.10/site-packages (from gluonts[torch]) (1.1
5.3)
Requirement already satisfied: PyYAML<8.0,>=5.4 in /opt/miniconda3/envs/econ
491_walmart_forcasting/lib/python3.10/site-packages (from lightning<2.5,>=
2.2.2->gluonts[torch]) (6.0.3)
Requirement already satisfied: lightning-utilities<2.0,>=0.10.0 in /opt/mini
conda3/envs/econ491_walmart_forcasting/lib/python3.10/site-packages (from li
ghtning<2.5,>=2.2.2->gluonts[torch]) (0.15.3)
Requirement already satisfied: packaging<25.0,>=20.0 in /opt/miniconda3/env
s/econ491_walmart_forcasting/lib/python3.10/site-packages (from lightning<
2.5,>=2.2.2->gluonts[torch]) (24.2)
Requirement already satisfied: torchmetrics<3.0,>=0.7.0 in /opt/miniconda3/e
nvs/econ491_walmart_forcasting/lib/python3.10/site-packages (from lightning<
2.5,>=2.2.2->gluonts[torch]) (1.9.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /opt/miniconda
3/envs/econ491_walmart_forcasting/lib/python3.10/site-packages (from fsspec
```

```
[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts[torch]) (3.13.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/miniconda3/env
s/econ491_walmart_forcasting/lib/python3.10/site-packages (from pandas<3,>=
1.0->gluonts[torch]) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/miniconda3/envs/econ491_
walmart_forcasting/lib/python3.10/site-packages (from pandas<3,>=1.0->gluont
s[torch]) (2026.1.post1)
Requirement already satisfied: tzdata>=2022.7 in /opt/miniconda3/envs/econ49
1_walmart_forcasting/lib/python3.10/site-packages (from pandas<3,>=1.0->gluo
nts[torch]) (2025.3)
Requirement already satisfied: annotated-types>=0.6.0 in /opt/miniconda3/env
s/econ491_walmart_forcasting/lib/python3.10/site-packages (from pydantic<3,>
=1.7->gluonts[torch]) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in /opt/miniconda3/env
s/econ491_walmart_forcasting/lib/python3.10/site-packages (from pydantic<3,>
=1.7->gluonts[torch]) (2.41.5)
Requirement already satisfied: typing-inspection>=0.4.2 in /opt/miniconda3/e
nvs/econ491_walmart_forcasting/lib/python3.10/site-packages (from pydantic<
3,>=1.7->gluonts[torch]) (0.4.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /opt/miniconda3/en
vs/econ491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=
4.0.0a0,!=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gl
uonts[torch]) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in /opt/miniconda3/envs/econ
491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts[tor
ch]) (1.4.0)
Requirement already satisfied: async-timeout<6.0,>=4.0 in /opt/miniconda3/en
vs/econ491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=
4.0.0a0,!=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gl
uonts[torch]) (5.0.1)
Requirement already satisfied: attrs>=17.3.0 in /opt/miniconda3/envs/econ491
_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=4.0.0a0,!=
4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts[torc
h]) (26.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /opt/miniconda3/envs/eco
n491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=4.0.0a
0,!=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts
[torch]) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /opt/miniconda3/envs/e
con491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=4.0.0a
0,!=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts
[torch]) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in /opt/miniconda3/envs/econ
491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts[tor
ch]) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /opt/miniconda3/envs/eco
n491_walmart_forcasting/lib/python3.10/site-packages (from aiohttp!=4.0.0a
0,!=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2->gluonts
[torch]) (1.23.0)
Requirement already satisfied: idna>=2.0 in /opt/miniconda3/envs/econ491_wal
mart_forcasting/lib/python3.10/site-packages (from yarl<2.0,>=1.17.0->aiohtt
p!=4.0.0a0,!=4.0.0a1->fsspec[http]<2026.0,>=2022.5.0->lightning<2.5,>=2.2.2-
>gluonts[torch]) (3.11)
Requirement already satisfied: six>=1.5 in /opt/miniconda3/envs/econ491_walm
```

```
art_forcasting/lib/python3.10/site-packages (from python-dateutil>=2.8.2->pa
ndas<3,>=1.0->gluonts[torch]) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/miniconda3/envs/ec
on491_walmart_forcasting/lib/python3.10/site-packages (from sympy>=1.13.3->t
orch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/miniconda3/envs/econ4
91_walmart_forcasting/lib/python3.10/site-packages (from jinja2->torch)
(3.0.2)
```

In [4]:
```
import pandas as pd
import numpy as np
from gluonts.dataset.common import ListDataset
from gluonts.torch.model.deepar import DeepAREstimator
from gluonts.evaluation import make_evaluation_predictions, Evaluator
import statsmodels.formula.api as smf
import gc
import os
```

In [5]:
```
# load data
os.chdir("/Users/conniezhang/Desktop/ECON491/data/")

sales_train = pd.read_csv("Sales Train Validation.csv")
calendar = pd.read_csv("Calendar (1).csv")
prices = pd.read_csv("sell_prices.csv")
```

In [6]:
```
# process the sales data

# Drop unused columns to save RAM immediately
sales_train = sales_train.drop(columns=["cat_id", "state_id"])

# Find all the day columns (d_1, d_2, etc.)
day_cols = [col for col in sales_train.columns if col.startswith('d_')]

# Melt the dataframe (equivalent to your long_sales in R)
long_sales = pd.melt(
    sales_train,
    id_vars=['item_id', 'dept_id', 'store_id'],
    value_vars=day_cols,
    var_name='d',
    value_name='sales'
)

# Clear old wide data from RAM
del sales_train
gc.collect()
```

Out[6]: 80

In [7]:
```
# prepare calendar data and join
calendar['d'] = 'd_' + (calendar.index + 1).astype(str)

# Keep only what we need
slim_calendar = calendar[['d', 'date', 'wm_yr_wk', 'wday', 'month', 'event_t
slim_calendar['date'] = pd.to_datetime(slim_calendar['date'])
```

```
# Merge sales and calendar (like your on="d" join)
final_df = pd.merge(long_sales, slim_calendar, on='d', how='left')

del long_sales, slim_calendar
gc.collect()
```

Out[7]: 0

In [8]:
```
# get average price for each item/store combination
# Get lifetime sales per item for weights
item_weights = final_df.groupby(['item_id', 'store_id', 'dept_id'])['sales']

# Merge prices with weights
prices_mapped = pd.merge(prices, item_weights, on=['item_id', 'store_id'])

# Calculate weighted average price per department per week
prices_mapped['weighted_price'] = prices_mapped['sell_price'] * prices_mappe
dept_prices = prices_mapped.groupby(['store_id', 'dept_id', 'wm_yr_wk']).app
    lambda x: x['weighted_price'].sum() / x['item_lifetime_sales'].sum()
).reset_index(name='avg_price')

del prices, prices_mapped, item_weights
gc.collect()
```

Out[8]: 0

In [9]:
```
# assemble model data
# Aggregate daily sales to store-dept level
model_data = final_df.groupby(['store_id', 'dept_id', 'date', 'wm_yr_wk', 'w

# Join prices
model_data = pd.merge(model_data, dept_prices, on=['store_id', 'dept_id', 'w

# Drop rows with missing prices
model_data = model_data.dropna(subset=['avg_price'])

# Create Dummy Variables for the Econometrics Model
model_data['day_of_week'] = model_data['wday'].astype(str) # Treat as catego
model_data['month_factor'] = model_data['month'].astype(str) # Treat as cate
model_data['is_holiday'] = np.where(model_data['event_type_1'].isna(), 0, 1)
```

In [10]:
```
# train/test split
split_date = pd.to_datetime("2016-04-24")
train_data = model_data[model_data['date'] <= split_date]
test_data = model_data[model_data['date'] > split_date].copy()
```

In [11]:
```
# run the simple benchmark model
# In Python's statsmodels, 'C()' tells it to treat a variable as a categoric
formula = "total_sales ~ avg_price + C(day_of_week) + C(month_factor) + is_h

# Fit the model
simple_model = smf.ols(formula=formula, data=train_data).fit()

# Print the beautiful Econometrics summary table!
print(simple_model.summary())
```

```
                            OLS Regression Results
==============================================================================
==
Dep. Variable:            total_sales   R-squared:                       0.1
23
Model:                            OLS   Adj. R-squared:                  0.1
22
Method:                 Least Squares   F-statistic:                     98
4.3
Date:                Mon, 06 Apr 2026   Prob (F-statistic):
0.00
Time:                        03:54:00   Log-Likelihood:            -1.0384e
+06
No. Observations:              133910   AIC:                         2.077e
+06
Df Residuals:                  133890   BIC:                         2.077e
+06
Df Model:                          19
Covariance Type:            nonrobust
==============================================================================
=============
                             coef    std err          t      P>|t|      [0.02
5      0.975]
------------------------------------------------------------------------------
-------------
Intercept              1263.4202      8.391    150.566      0.000    1246.97
4    1279.867
C(day_of_week)[T.2]      -3.4308      5.793     -0.592      0.554     -14.78
4       7.923
C(day_of_week)[T.3]    -121.9665      5.796    -21.042      0.000    -133.32
7    -110.606
C(day_of_week)[T.4]    -159.1549      5.772    -27.576      0.000    -170.46
7    -147.843
C(day_of_week)[T.5]    -163.9375      5.775    -28.389      0.000    -175.25
6    -152.619
C(day_of_week)[T.6]    -160.5140      5.780    -27.771      0.000    -171.84
2    -149.186
C(day_of_week)[T.7]    -104.8022      5.770    -18.163      0.000    -116.11
1     -93.493
C(month_factor)[T.10]    15.3010      7.628      2.006      0.045       0.35
0      30.252
C(month_factor)[T.11]    -7.6814      7.693     -0.999      0.318     -22.75
9       7.396
C(month_factor)[T.12]   -10.5144      7.630     -1.378      0.168     -25.47
0       4.441
C(month_factor)[T.2]     10.2479      7.465      1.373      0.170      -4.38
4      24.880
C(month_factor)[T.3]      6.0649      7.300      0.831      0.406      -8.24
3      20.373
C(month_factor)[T.4]      4.3948      7.417      0.593      0.553     -10.14
2      18.932
C(month_factor)[T.5]    -20.5718      7.628     -2.697      0.007     -35.52
3      -5.621
C(month_factor)[T.6]     12.3148      7.692      1.601      0.109      -2.76
1      27.391
C(month_factor)[T.7]     17.3144      7.631      2.269      0.023       2.35
```

```
8      32.271
C(month_factor)[T.8]     26.9269      7.638      3.525      0.000      11.95
6      41.898
C(month_factor)[T.9]     22.7449      7.699      2.954      0.003       7.65
6      37.834
avg_price              -209.9684      1.623   -129.372      0.000    -213.14
9    -206.787
is_holiday              -25.5393      5.775     -4.423      0.000     -36.85
8     -14.221
==============================================================================
==
Omnibus:                    51438.959   Durbin-Watson:                   0.0
77
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           240219.6
59
Skew:                           1.840   Prob(JB):
0.00
Kurtosis:                       8.432   Cond. No.                         4
4.2
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
```

In [12]:

```
# reshaping data for deepAR
ts_data = model_data.pivot_table(
    index='date',
    columns=['store_id', 'dept_id'],
    values='total_sales',
    aggfunc='sum'
).fillna(0)

ts_data.index = pd.to_datetime(ts_data.index)
ts_data = ts_data.asfreq('D')
ts_data.iloc[:5, :4]
# ts_data.columns.get_level_values('store_id').unique()
```

Out[12]:

store_id	CA_1			
dept_id	FOODS_1	FOODS_2	FOODS_3	HOBBIES_1
date				
2011-01-29	297	674	2268	528
2011-01-30	284	655	2198	489
2011-01-31	214	396	1398	409
2011-02-01	175	476	1607	383
2011-02-02	182	354	1496	263

In [13]:

```
# creating train and test datasets
prediction_length = 28 # We are forecasting exactly 28 days
```

```
split_date = pd.to_datetime("2016-04-24")

# Split the time series
train_ts = ts_data[ts_data.index <= split_date]
test_ts = ts_data # In GluonTS, the test set includes the historical context

# Convert to GluonTS ListDataset format
train_ds = ListDataset(
    [{"start": train_ts.index[0], "target": train_ts[col].values} for col in
    freq="D"
)

test_ds = ListDataset(
    [{"start": test_ts.index[0], "target": test_ts[col].values} for col in t
    freq="D"
)
```

In [14]:

```
# initializing deepAr neural nework
estimator = DeepAREstimator(
    freq="D",
    prediction_length=prediction_length,
    context_length=prediction_length * 2,
    num_layers=2,
    hidden_size=32,
    trainer_kwargs={
        "max_epochs":30,
        "accelerator": "cpu",
        "enable_progress_bar": False
    }
)
```

In [15]:

```
%%capture
# traning model
predictor = estimator.train(train_ds)
print("training complete")
```

```
GPU available: True (mps), used: False
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs

  | Name  | Type        | Params | Mode  | In sizes
| Out sizes
--------------------------------------------------------------------------------
-----------------------------------------------------
0 | model | DeepARModel | 17.6 K | train | [[1, 1], [1, 1], [1, 1148, 4],
[1, 1148], [1, 1148], [1, 28, 4]] | [1, 100, 28]
--------------------------------------------------------------------------------
-----------------------------------------------------
17.6 K    Trainable params
0         Non-trainable params
17.6 K    Total params
0.071     Total estimated model params size (MB)
11        Modules in train mode
0         Modules in eval mode
Epoch 0, global step 50: 'train_loss' reached 6.29722 (best 6.29722), saving
model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_23/
checkpoints/epoch=0-step=50.ckpt' as top 1
Epoch 1, global step 100: 'train_loss' reached 5.62223 (best 5.62223), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=1-step=100.ckpt' as top 1
Epoch 2, global step 150: 'train_loss' reached 5.57516 (best 5.57516), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=2-step=150.ckpt' as top 1
Epoch 3, global step 200: 'train_loss' reached 5.44528 (best 5.44528), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=3-step=200.ckpt' as top 1
Epoch 4, global step 250: 'train_loss' reached 5.32533 (best 5.32533), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=4-step=250.ckpt' as top 1
Epoch 5, global step 300: 'train_loss' reached 5.29539 (best 5.29539), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=5-step=300.ckpt' as top 1
Epoch 6, global step 350: 'train_loss' reached 5.26447 (best 5.26447), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=6-step=350.ckpt' as top 1
Epoch 7, global step 400: 'train_loss' was not in top 1
Epoch 8, global step 450: 'train_loss' reached 5.25150 (best 5.25150), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=8-step=450.ckpt' as top 1
Epoch 9, global step 500: 'train_loss' reached 5.22669 (best 5.22669), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
3/checkpoints/epoch=9-step=500.ckpt' as top 1
Epoch 10, global step 550: 'train_loss' reached 5.18490 (best 5.18490), savi
ng model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_
23/checkpoints/epoch=10-step=550.ckpt' as top 1
Epoch 11, global step 600: 'train_loss' was not in top 1
Epoch 12, global step 650: 'train_loss' was not in top 1
Epoch 13, global step 700: 'train_loss' was not in top 1
Epoch 14, global step 750: 'train_loss' was not in top 1
Epoch 15, global step 800: 'train_loss' was not in top 1
```

```
Epoch 16, global step 850: 'train_loss' was not in top 1
Epoch 17, global step 900: 'train_loss' was not in top 1
Epoch 18, global step 950: 'train_loss' was not in top 1
Epoch 19, global step 1000: 'train_loss' was not in top 1
Epoch 20, global step 1050: 'train_loss' was not in top 1
Epoch 21, global step 1100: 'train_loss' was not in top 1
Epoch 22, global step 1150: 'train_loss' reached 5.17086 (best 5.17086), sav
ing model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version
_23/checkpoints/epoch=22-step=1150.ckpt' as top 1
Epoch 23, global step 1200: 'train_loss' was not in top 1
Epoch 24, global step 1250: 'train_loss' was not in top 1
Epoch 25, global step 1300: 'train_loss' was not in top 1
Epoch 26, global step 1350: 'train_loss' was not in top 1
Epoch 27, global step 1400: 'train_loss' was not in top 1
Epoch 28, global step 1450: 'train_loss' was not in top 1
Epoch 29, global step 1500: 'train_loss' was not in top 1
`Trainer.fit` stopped: `max_epochs=30` reached.
```

In [16]:

```
forecast_it, ts_it = make_evaluation_predictions(
    dataset=test_ds,
    predictor=predictor,
    num_samples=100,  # DeepAR generates 100 possible futures to find the pr
)

forecasts = list(forecast_it)
tss = list(ts_it)

# Calculate RMSE and MAPE automatically using GluonTS Evaluator
evaluator = Evaluator(quantiles=[0.5]) # We evaluate the median forecast
agg_metrics, item_metrics = evaluator(tss, forecasts)

print("-" * 40)
print(f"DeepAR Overall RMSE: {agg_metrics['RMSE']:.2f}")
print(f"DeepAR Overall MAPE: {agg_metrics['MAPE'] * 100:.2f}%")
print("-" * 40)
```

```
Running evaluation: 70it [00:00, 813.19it/s]
----------------------------------------
DeepAR Overall RMSE: 99.81
DeepAR Overall MAPE: 13.96%
----------------------------------------

/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
```

In [17]:
```
#check overfitting

# 1. Ask the model to predict the final 28 days of the PAST (Training data)
forecast_it_train, ts_it_train = make_evaluation_predictions(
    dataset=train_ds,  # <--- Notice we are using the TRAIN set here!
    predictor=predictor,
    num_samples=100,
)

# 2. Calculate the metrics for the past
agg_metrics_train, _ = evaluator(list(ts_it_train), list(forecast_it_train))

# 3. Print the Apples-to-Apples Comparison
print("-" * 40)
print(f"TRAIN MAPE (The Past):   {agg_metrics_train['MAPE'] * 100:.2f}%")
print(f"TEST MAPE  (The Future): 13.96%") # From your earlier evaluation
print("-" * 40)
```

```
Running evaluation: 70it [00:00, 874.92it/s]
----------------------------------------
TRAIN MAPE (The Past):   13.95%
TEST MAPE  (The Future): 13.96%
----------------------------------------
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
```

In [18]:
```
# We pivot the price data exactly like we did the sales data
price_data = model_data.pivot_table(
    index='date',
    columns=['store_id', 'dept_id'],
    values='avg_price',
    aggfunc='mean'
)

# If a store was closed, sales were 0. But the price of an item doesn't drop
# 'ffill' (forward fill) carries yesterday's price forward for any missing d
price_data = price_data.fillna(method='ffill').fillna(0)

price_data.index = pd.to_datetime(price_data.index)
price_data = price_data.asfreq('D')
```

In [19]:
```
# rebuilding datases with economic features
```

```
train_price = price_data[price_data.index <= split_date]
test_price = price_data

# Injecting the price timeline into the neural network's dataset
train_ds_econ = ListDataset(
    [
        {
            "start": train_ts.index[0],
            "target": train_ts[col].values,
            "feat_dynamic_real": [train_price[col].values] # <--- PRICE ADDE
        }
        for col in train_ts.columns
    ],
    freq="D"
)

test_ds_econ = ListDataset(
    [
        {
            "start": test_ts.index[0],
            "target": test_ts[col].values,
            "feat_dynamic_real": [test_price[col].values]
        }
        for col in test_ts.columns
    ],
    freq="D"
)
```

In [20]:

```
# initizling deepAr neural nework with economic features
estimator_econ = DeepAREstimator(
    freq="D",
    prediction_length=prediction_length,
    context_length=prediction_length * 2,
    num_feat_dynamic_real=1,      # <--- We are giving it 1 feature: 'avg_pr
    num_layers=2,
    hidden_size=32,
    trainer_kwargs={
        "max_epochs":30,
        "accelerator": "cpu",
        "enable_progress_bar": False
    }
)
```

In [21]:

```
%%capture
# traning econ model
predictor_econ = estimator_econ.train(train_ds_econ)
print("Training Complete!")
```

```
GPU available: True (mps), used: False
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs

  | Name  | Type        | Params | Mode  | In sizes
| Out sizes
--------------------------------------------------------------------------------
-----------------------------------------------------
0 | model | DeepARModel | 17.8 K | train | [[1, 1], [1, 1], [1, 1148, 5],
[1, 1148], [1, 1148], [1, 28, 5]] | [1, 100, 28]
--------------------------------------------------------------------------------
-----------------------------------------------------
17.8 K    Trainable params
0         Non-trainable params
17.8 K    Total params
0.071     Total estimated model params size (MB)
11        Modules in train mode
0         Modules in eval mode
Epoch 0, global step 50: 'train_loss' reached 6.33749 (best 6.33749), saving
model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_24/
checkpoints/epoch=0-step=50.ckpt' as top 1
Epoch 1, global step 100: 'train_loss' reached 5.59313 (best 5.59313), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=1-step=100.ckpt' as top 1
Epoch 2, global step 150: 'train_loss' reached 5.52855 (best 5.52855), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=2-step=150.ckpt' as top 1
Epoch 3, global step 200: 'train_loss' reached 5.40980 (best 5.40980), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=3-step=200.ckpt' as top 1
Epoch 4, global step 250: 'train_loss' reached 5.32687 (best 5.32687), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=4-step=250.ckpt' as top 1
Epoch 5, global step 300: 'train_loss' reached 5.26373 (best 5.26373), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=5-step=300.ckpt' as top 1
Epoch 6, global step 350: 'train_loss' was not in top 1
Epoch 7, global step 400: 'train_loss' reached 5.23091 (best 5.23091), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=7-step=400.ckpt' as top 1
Epoch 8, global step 450: 'train_loss' was not in top 1
Epoch 9, global step 500: 'train_loss' reached 5.21943 (best 5.21943), savin
g model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_2
4/checkpoints/epoch=9-step=500.ckpt' as top 1
Epoch 10, global step 550: 'train_loss' was not in top 1
Epoch 11, global step 600: 'train_loss' reached 5.20011 (best 5.20011), savi
ng model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_
24/checkpoints/epoch=11-step=600.ckpt' as top 1
Epoch 12, global step 650: 'train_loss' reached 5.19086 (best 5.19086), savi
ng model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_
24/checkpoints/epoch=12-step=650.ckpt' as top 1
Epoch 13, global step 700: 'train_loss' was not in top 1
Epoch 14, global step 750: 'train_loss' was not in top 1
Epoch 15, global step 800: 'train_loss' was not in top 1
```

```
Epoch 16, global step 850: 'train_loss' was not in top 1
Epoch 17, global step 900: 'train_loss' reached 5.15065 (best 5.15065), savi
ng model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version_
24/checkpoints/epoch=17-step=900.ckpt' as top 1
Epoch 18, global step 950: 'train_loss' was not in top 1
Epoch 19, global step 1000: 'train_loss' was not in top 1
Epoch 20, global step 1050: 'train_loss' was not in top 1
Epoch 21, global step 1100: 'train_loss' was not in top 1
Epoch 22, global step 1150: 'train_loss' was not in top 1
Epoch 23, global step 1200: 'train_loss' was not in top 1
Epoch 24, global step 1250: 'train_loss' was not in top 1
Epoch 25, global step 1300: 'train_loss' was not in top 1
Epoch 26, global step 1350: 'train_loss' was not in top 1
Epoch 27, global step 1400: 'train_loss' was not in top 1
Epoch 28, global step 1450: 'train_loss' reached 5.13072 (best 5.13072), sav
ing model to '/Users/conniezhang/Desktop/ECON491/data/lightning_logs/version
_24/checkpoints/epoch=28-step=1450.ckpt' as top 1
Epoch 29, global step 1500: 'train_loss' was not in top 1
`Trainer.fit` stopped: `max_epochs=30` reached.
```

In [22]:

```python
# evalueating advanced model with price data
forecast_it, ts_it = make_evaluation_predictions(
    dataset=test_ds_econ,
    predictor=predictor_econ,
    num_samples=100,
)

agg_metrics, item_metrics = evaluator(list(ts_it), list(forecast_it))

print("-" * 40)
print(f"DeepAR (w/ Price) RMSE: {agg_metrics['RMSE']:.2f}")
print(f"DeepAR (w/ Price) MAPE: {agg_metrics['MAPE'] * 100:.2f}%")
print("-" * 40)
```

```
Running evaluation: 70it [00:00, 773.75it/s]
----------------------------------------
DeepAR (w/ Price) RMSE: 94.87
DeepAR (w/ Price) MAPE: 14.00%
----------------------------------------

/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
```

In [23]:

```python
forecast_it, ts_it = make_evaluation_predictions(
```

```
    dataset=test_ds,
    predictor=predictor,
    num_samples=100,
)

forecasts_base = list(forecast_it)
tss_base = list(ts_it)
```

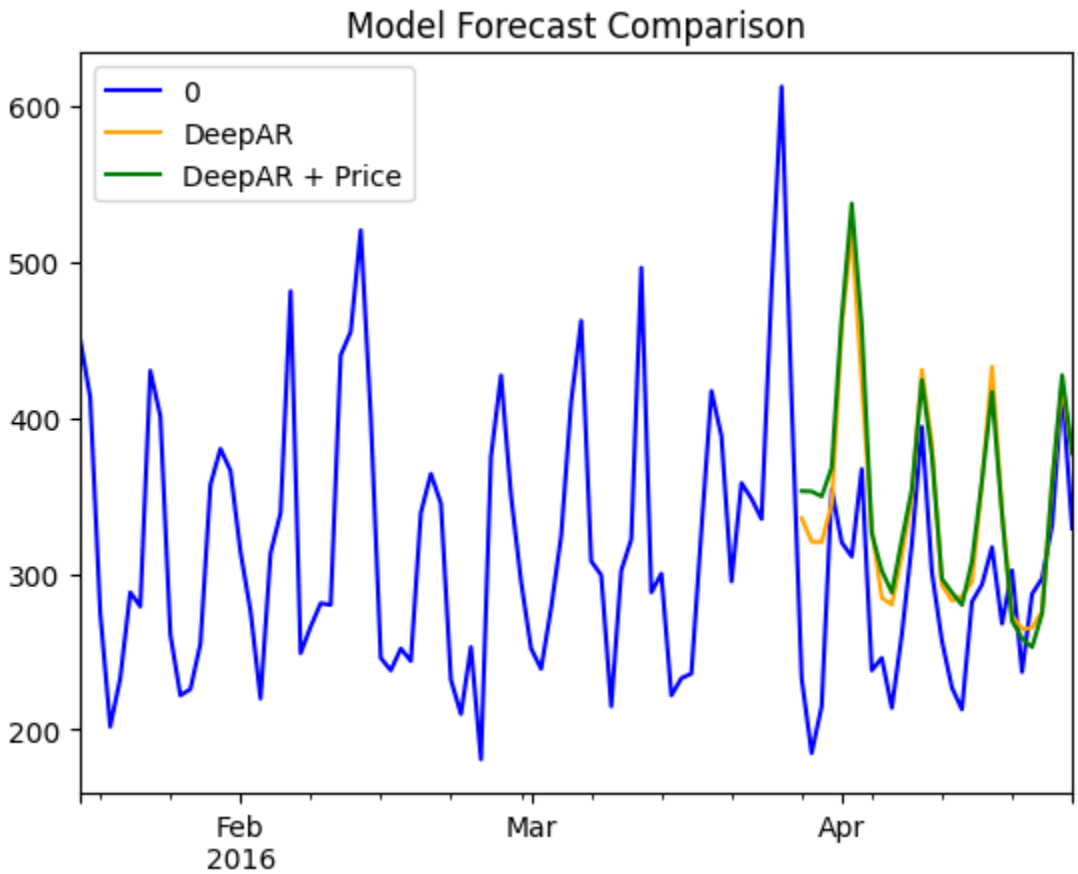
In [24]:
```
#check overfitting

# 1. Ask the model to predict the final 28 days of the PAST (Training data)
forecast_it_train, ts_it_train = make_evaluation_predictions(
    dataset=train_ds_econ,  # <--- Notice we are using the TRAIN set here!
    predictor=predictor_econ,
    num_samples=100,
)

# 2. Calculate the metrics for the past
agg_metrics_train, _ = evaluator(list(ts_it_train), list(forecast_it_train))

# 3. Print the Apples-to-Apples Comparison
print("-" * 40)
print(f"TRAIN MAPE (The Past):   {agg_metrics_train['MAPE'] * 100:.2f}%")
print(f"TEST MAPE  (The Future): 14.00%")
print("-" * 40)
```

```
Running evaluation: 70it [00:00, 747.64it/s]
----------------------------------------
TRAIN MAPE (The Past):   14.06%
TEST MAPE  (The Future): 14.00%
----------------------------------------
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
/opt/miniconda3/envs/econ491_walmart_forcasting/lib/python3.10/site-package
s/gluonts/json.py:102: UserWarning: Using `json`-module for json-handling. C
onsider installing one of `orjson`, `ujson` to speed up serialization and de
serialization.
  warnings.warn(
```

In [25]:
```
forecast_it, ts_it = make_evaluation_predictions(
    dataset=test_ds_econ,
    predictor=predictor_econ,
    num_samples=100,
)

forecasts_econ = list(forecast_it)
tss_econ = list(ts_it)
```

In [26]:
```
import matplotlib.pyplot as plt

# Choose first time series
ts = tss_base[0]

forecast_base = forecasts_base[0]
forecast_econ = forecasts_econ[0]

plt.figure(figsize=(12,6))

# Actual values
ts[-100:].plot(color='blue', label="Actual")

# Base model prediction (median)
plt.plot(forecast_base.index, forecast_base.quantile(0.5), color='orange', l

# Econ model prediction (median)
plt.plot(forecast_econ.index, forecast_econ.quantile(0.5), color='green', la

plt.title("Model Forecast Comparison")
plt.legend()
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```



In [ ]: